Two Simulators, One Controller: The Stage 1 and Stage 2 Training Recipe in OmniReset

Tri Wahyu Guntara
Training notes on the UWLab OmniReset code base

This note is a companion to the controller tutorial *From a Desired End-Effector Pose to Joint Torques* in the same folder. That tutorial explains how the relative Cartesian operational-space controller (OSC) turns a policy action into joint torques [1]. This note explains what happens to those torques afterwards, and how the training recipe is organised around that.

1 The Two Stages at a Glance

The OmniReset insertion policy [2] is trained in two stages. Stage 1 trains the policy from scratch in a deliberately forgiving simulator. Stage 2 loads the converged stage 1 checkpoint and continues training while the simulator is gradually made to behave like the physical robot. The controller, the reward, the observations and the network are identical in both stages. What changes is the plant that the controller acts on, together with the gains and the action scale that the controller is given. Figure 1 summarises the recipe.

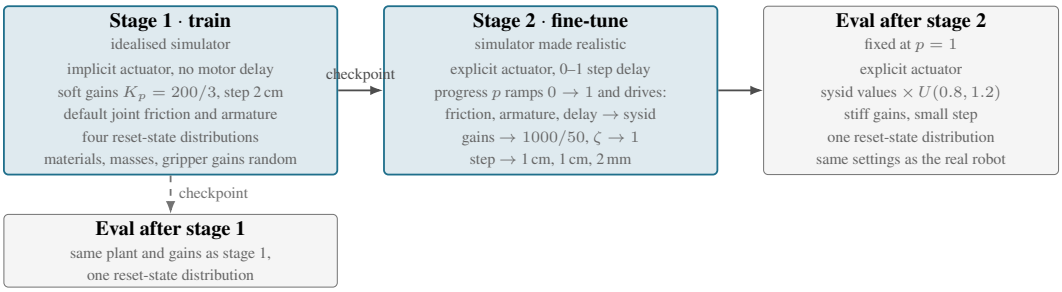


Figure 1: The training recipe. Stage 2 starts from the stage 1 checkpoint and ramps a single progress variable from 0 to 1. At 0 the simulator is the stage 1 simulator except for the actuator model. At 1 it is the plant that the evaluation environment and the physical robot use.

Table 1 collects the settings that differ between the four environments. The rest of this note explains each row.

Table 1: What each environment sets. Gains are written as translation / rotation. $U(a, b)$ is a uniform draw per environment at every reset.

Setting	Stage 1 train	Stage 2 fine-tune	Eval after stage 1	Eval after stage 2
Arm actuator model	implicit	explicit, delayed PD	implicit	explicit, delayed PD
Motor delay	none	0 to $\mathrm{round}(p)$ steps	none	0 to 1 steps
Joint friction, armature	asset defaults	defaults → sysid $\times\, U(0.8, 1.2)$	asset defaults	sysid $\times\, U(0.8, 1.2)$
OSC stiffness K_p	200 / 3	200 / 3 → 1000 / 50, $\times\, U(0.8, 1.2)$	200 / 3	1000 / 50 $\times\, U(0.8, 1.2)$
OSC damping ratio ζ	3 / 1 (UR5e), 0 (G1-D)	→ 1 / 1, $\times\, U(0.8, 1.2)$	as training	1 / 1 $\times\, U(0.8, 1.2)$
Action scale x, y, z	2, 2, 2 cm	→ 1, 1, 0.2 cm	2, 2, 2 cm	1, 1, 0.2 cm
Action scale rotation	0.02, 0.02, 0.2 rad	unchanged	unchanged	unchanged
Reset-state distributions	four, equal probability	four, equal probability	one	one
Curriculum	none	two success-gated progress variables	none	none

2 Where the Torque Goes: The Actuator Layer

Every physics step, the controller computes a torque vector for the arm joints and hands it to the articulation as an *effort target*. That target does not reach the physics engine directly. In Isaac Lab [3] an *actuator model* sits between the action term and the simulator, and the actuator model decides what the simulator finally receives. The two training stages use two different actuator models, so this layer is the first thing to understand.

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/mdp/actions/task_space_actions.py

lines 177–180, 192–194

```
177 # PD control: tau = J^T @ (Kp * err + Kd * (-vel))
178 vel_error = -ee_vel
179 task_force = self._kp * pose_error + self._kd * vel_error
180 joint_torques = torch.bmm(jacobian.transpose(-1, -2), task_force.unsqueeze(-1)).squeeze(-1)
    :
192 joint_torques = torch.clamp(joint_torques, -self._torque_max, self._torque_max)
193
194 self._asset.set_joint_effort_target(joint_torques, joint_ids=self._joint_ids)
```

2.1 The joint drive that PhysX offers

PhysX can run a proportional–derivative (PD) drive on every joint. Given a position target q^*, a velocity target $\dot{q}^*$ and a feed-forward effort τ_{ff}, the drive applies

$$\tau = k_s\,(q^* - q) + k_d\,(\dot{q}^* - \dot{q}) + \tau_{\text{ff}}, \tag{1}$$

where k_s is the joint stiffness and k_d the joint damping. The important property is *how* PhysX evaluates this expression. The solver treats the drive as a constraint and solves for the velocity at the end of the step, so the damping term is evaluated with the new velocity rather than the old one. This is an implicit integration, and an implicit spring–damper is stable for any positive gain and any step size. Section 2.5 shows why that matters.

2.2 The implicit actuator

Isaac Lab's `ImplicitActuator` is a thin wrapper around that PhysX drive. Its `compute` method receives the targets, writes some telemetry, and returns the targets *unchanged*. The stiffness and damping it carries are written into the PhysX drive once at start-up, and PhysX does the rest.

IsaacLab/source/isaaclab/isaaclab/actuators/actuator_pd.py lines 199–201, 221–227

```
199 def compute(
200     self, control_action: ArticulationActions, joint_pos: torch.Tensor, joint_vel: torch.Tensor
201 ) -> ArticulationActions:
    :
221     # store approximate torques for reward computation
222     error_pos = control_action.joint_positions - joint_pos
223     error_vel = control_action.joint_velocities - joint_vel
224     self.computed_effort = self.stiffness * error_pos + self.damping * error_vel + control_action.
          joint_efforts
225     # clip the torques based on the motor limits
226     self.applied_effort = self._clip_effort(self.computed_effort)
227     return control_action
```

The telemetry on lines 222 to 226 is an estimate for logging and rewards. It is not what PhysX applies, because PhysX evaluates the same expression implicitly.

2.3 Explicit actuators

Every other actuator in Isaac Lab is *explicit*. The base class is `IdealPDActuator`. It evaluates the same PD expression itself, in Python, with the current joint state, clips the result to the actuator's own effort limit, and then does something decisive on lines 332 to 334: it moves the computed effort

into the effort slot and erases the position and velocity targets. PhysX therefore receives a pure effort, held constant for one physics step, and its internal drive never engages.

IsaacLab/source/isaaclab/isaaclab/actuators/actuator_pd.py lines 321–335

```
321 def compute(
322     self, control_action: ArticulationActions, joint_pos: torch.Tensor, joint_vel: torch.Tensor
323 ) -> ArticulationActions:
324     # compute errors
325     error_pos = control_action.joint_positions - joint_pos
326     error_vel = control_action.joint_velocities - joint_vel
327     # calculate the desired joint torques
328     self.computed_effort = self.stiffness * error_pos + self.damping * error_vel + control_action.
        joint_efforts
329     # clip the torques based on the motor limits
330     self.applied_effort = self._clip_effort(self.computed_effort)
331     # set the computed actions back into the control action
332     control_action.joint_efforts = self.applied_effort
333     control_action.joint_positions = None
334     control_action.joint_velocities = None
335     return control_action
```

Because the Python code can do anything to the targets before they reach PhysX, explicit actuators are where motor imperfections are modelled. `DelayedPDActuator` is the one used here. It draws a delay in physics steps for every environment at reset, between `min_delay` and `max_delay`, and pushes the incoming targets through a circular buffer of that length before evaluating the PD law. With the configuration used in OmniReset, the delay is zero or one physics step.

IsaacLab/source/isaaclab/isaaclab/actuators/actuator_pd.py lines 486–497. 503–512

```
486     # set a new random delay for environments in env_ids
487     time_lags = torch.randint(
488         low=self.cfg.min_delay,
489         high=self.cfg.max_delay + 1,
490         size=(num_envs,),
491         dtype=torch.int,
492         device=self._device,
493     )
494     # set delays
495     self.positions_delay_buffer.set_time_lag(time_lags, env_ids)
496     self.velocities_delay_buffer.set_time_lag(time_lags, env_ids)
497     self.efforts_delay_buffer.set_time_lag(time_lags, env_ids)
    :
503 def compute(
504     self, control_action: ArticulationActions, joint_pos: torch.Tensor, joint_vel: torch.Tensor
505 ) -> ArticulationActions:
506     # apply delay based on the delay the model for all the setpoints
507     control_action.joint_positions = self.positions_delay_buffer.compute(control_action.
        joint_positions)
508     control_action.joint_velocities = self.velocities_delay_buffer.compute(control_action.
        joint_velocities)
509     control_action.joint_efforts = self.efforts_delay_buffer.compute(control_action.joint_efforts)
510     # compte actuator model
511     return super().compute(control_action, joint_pos, joint_vel)
```

2.4 With zero gains, both actuators are a pipe

The OmniReset arm configurations set the actuator stiffness and damping to zero in *both* variants. With $k_s = k_d = 0$, Eq. (1) reduces to $\tau = \tau_{\text{ff}}$, and τ_{ff} is exactly the torque the OSC produced. Neither actuator is doing position control. The only closed loop in the system is the task-space PD law inside the OSC, and the actuator is the pipe through which its torque flows. Figure 2 shows the two pipes side by side.

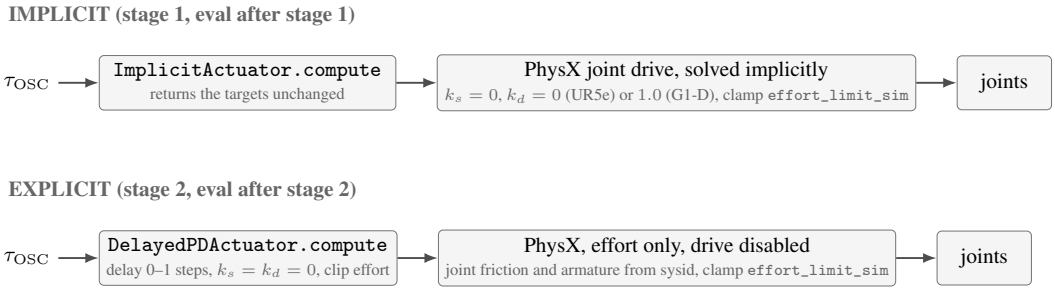


Figure 2: The two actuator pipes. In both, the actuator gains are zero and the OSC torque passes through. The explicit pipe adds a delay of zero or one physics step and a software effort clip, and in stage 2 the joints behind it carry the friction and rotor inertia identified on the real arm.

The configurations below are the two variants for each robot. Note that the G1-D implicit actuator carries a joint damping of 1.0 N m s/rad while every other entry is zero. The reason is the subject of the next section.

source/uwlab_assets/uwlab_assets/robots/ur5e_robotiq_gripper/ur5e_robotiq_2f85_gripper.py lines 165–175

```python
IMPLICIT_UR5E_ROBOTIQ_2F85 = UR5E_ARTICULATION.copy()  # type: ignore
IMPLICIT_UR5E_ROBOTIQ_2F85.actuators = {
    "arm": ImplicitActuatorCfg(
        joint_names_expr=["shoulder.*", "elbow.*", "wrist.*"],
        stiffness=0.0,
        damping=0.0,
        effort_limit_sim=UR5E_EFFORT_LIMITS,
        velocity_limit_sim=UR5E_VELOCITY_LIMITS,
    ),
    "gripper": ROBOTIQ_2F85.actuators["gripper"],
}
```

source/uwlab_assets/uwlab_assets/robots/ur5e_robotiq_gripper/ur5e_robotiq_2f85_gripper.py lines 149–163

```python
EXPLICIT_UR5E_ROBOTIQ_2F85 = UR5E_ARTICULATION.copy()  # type: ignore
EXPLICIT_UR5E_ROBOTIQ_2F85.actuators = {
    "arm": DelayedPDActuatorCfg(
        joint_names_expr=["shoulder.*", "elbow.*", "wrist.*"],
        stiffness=0.0,
        damping=0.0,
        effort_limit=UR5E_EFFORT_LIMITS,
        effort_limit_sim=UR5E_EFFORT_LIMITS,
        velocity_limit=UR5E_VELOCITY_LIMITS,
        velocity_limit_sim=UR5E_VELOCITY_LIMITS,
        min_delay=0,
        max_delay=1,
    ),
    "gripper": ROBOTIQ_2F85.actuators["gripper"],
}
```

source/uwlab_assets/uwlab_assets/robots/unitree_g1d_dex1/g1d_dex1.py lines 137–152

```python
IMPLICIT_G1D_DEX1 = G1D_DEX1_ARTICULATION.copy()  # type: ignore
IMPLICIT_G1D_DEX1.actuators = {
    "arm": ImplicitActuatorCfg(
        joint_names_expr=G1D_ARM_JOINT_REGEX,
        stiffness=0.0,
        # Joint damping lives here (PhysX applies it implicitly, stable at any gain) instead of in the OSC: the
        # wrist links weigh ~85 g (I ~ 5e-4 kg m^2 about the roll axis), and an explicit task-space damping torque
        # of a few N m s/rad at 120 Hz overshoots the velocity each substep and diverges within a handful of
        # substeps (verified with scripts_v2/tools/debug_reset_env-style probes on 2026-10-26). The UR5e gets away
        # with explicit damping only because its wrist links are ~10x heavier.
        damping=1.0,
        effort_limit_sim=G1D_EFFORT_LIMITS,
        velocity_limit_sim=G1D_VELOCITY_LIMITS,
    ),
    "gripper": DEX1_GRIPPER_ACTUATOR,
```

```
}
```

source/uwlab_assets/uwlab_assets/robots/unitree_g1d_dex1/g1d_dex1.py lines 121–135

```
EXPLICIT_G1D_DEX1 = G1D_DEX1_ARTICULATION.copy()  # type: ignore
EXPLICIT_G1D_DEX1.actuators = {
    "arm": DelayedPDActuatorCfg(
        joint_names_expr=G1D_ARM_JOINT_REGEX,
        stiffness=0.0,
        damping=0.0,
        effort_limit=G1D_EFFORT_LIMITS,
        effort_limit_sim=G1D_EFFORT_LIMITS,
        velocity_limit=G1D_VELOCITY_LIMITS,
        velocity_limit_sim=G1D_VELOCITY_LIMITS,
        min_delay=0,
        max_delay=1,
    ),
    "gripper": DEX1_GRIPPER_ACTUATOR,
}
```

Three small differences remain between the two pipes, and they are worth naming because they are the whole difference between the stage 1 plant and the stage 2 plant as far as the actuator is concerned.

1. **Delay.** The explicit pipe holds each torque for zero or one physics step before it applies. At 120 Hz that is up to 8.3 ms, which is the order of the command latency of a real joint controller.
2. **Where the effort clamp lives.** The implicit pipe clamps inside the solver with `effort_limit_sim`. The explicit pipe clamps in Python with `effort_limit` first, and the solver clamp still applies afterwards. The numbers are the same, so the effect is the same.
3. **Joint damping on the G1-D.** The implicit G1-D arm has $1.0\,\mathrm{N\,m\,s/rad}$ of joint damping inside the PhysX drive. The explicit G1-D arm has none. That is a real difference in plant behaviour, and it is discussed in Section 7.

2.5 Implicit versus explicit damping, and the light wrist

Why does any of this matter if the gains are zero anyway? Because the OSC itself contains a damping term, $K_d\,J\dot{q}$, and that term is evaluated explicitly: the controller computes it once from the current velocity and holds it for a whole physics step. Consider a single joint with inertia I about its axis and a damping torque $-c\,\omega$ applied this way. Over one step of length h the velocity updates as

$$\begin{aligned} \omega_{n+1} &= \left(1 - \frac{c\,h}{I}\right)\omega_n \quad \text{(explicit)}, \\ \omega_{n+1} &= \frac{\omega_n}{1 + c\,h/I} \qquad \text{(implicit)}. \end{aligned} \tag{2}$$

The implicit update shrinks the velocity for every $c > 0$. The explicit update shrinks it only while $c\,h/I < 2$. Beyond that bound each step overshoots zero by more than it started with, and the velocity grows without limit. The bound is therefore

$$c < \frac{2\,I}{h}. \tag{3}$$

The numbers for the G1-D wrist are unforgiving. The wrist links weigh about 85 g, with an inertia of roughly $5 \times 10^{-4}\,\mathrm{kg\,m^2}$ about the roll axis, and the physics step is $h = 1/120\,\mathrm{s}$. The bound is then about $0.12\,\mathrm{N\,m\,s/rad}$. The rotational task-space damping of the OSC is $K_d = 2\zeta\sqrt{K_p}$, which is about $3.5\,\mathrm{N\,m\,s/rad}$ with the UR5e training gains and about $14\,\mathrm{N\,m\,s/rad}$ with the evaluation gains. When the wrist roll axis is aligned with the task rotation axis, the transposed Jacobian maps that task damping almost one to one onto the wrist joint. Both values exceed the bound by more than an order of magnitude.

What the code says. The comment in the G1-D implicit actuator configuration records the empirical version of this argument: an explicit task-space damping torque of a few N m s/rad at 120 Hz

overshoots the velocity each substep and diverges within a handful of substeps, verified with probe scripts on 26 October 2026. The isolated-link bound of Eq. (3) is a simplification. The effective inertia seen by a joint includes everything downstream of it, and PhysX substeps the solve, so the real threshold is higher than 0.12. The order of magnitude is what matters, and it explains why the UR5e, whose wrist links are about ten times heavier and carry a 0.5 kg gripper, tolerates the same damping that destabilises the G1-D.

The G1-D configuration therefore makes two linked choices. The OSC damping ratio is set to zero on all six axes, so the controller applies no explicit damping at all. The damping is moved into the implicit actuator as a joint damping of $1.0\,\mathrm{N\,m\,s/rad}$, which PhysX integrates implicitly and which is stable regardless of its value. The gains are otherwise identical to the UR5e recipe.

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/config/g1d_dex1/actions.py lines 27–50

```
G1D_DEX1_RELATIVE_OSC = RelCartesianOSCActionCfg(
    asset_name="robot",
    joint_names=G1D_ARM_JOINT_NAMES,
    body_name=G1D_EE_BODY_NAME,
    scale_xyz_axisangle=(0.02, 0.02, 0.02, 0.02, 0.02, 0.2),
    motion_stiffness=(200.0, 200.0, 200.0, 3.0, 3.0, 3.0),
    # No explicit task-space damping: it is unstable on the light G1-D wrist links; the damping is provided
    # implicitly by the arm actuator (see IMPLICIT_G1D_DEX1 in uwlab_assets).
    motion_damping_ratio=(0.0, 0.0, 0.0, 0.0, 0.0, 0.0),
    torque_limit=G1D_TORQUE_LIMIT,
    jacobian_fn=G1D_JACOBIAN_FN,
    # 7-DoF arm: hold the default elbow configuration in the null space of the task
    nullspace_stiffness=2.0,
    nullspace_damping_ratio=1.0,
    nullspace_joint_pos=G1D_DEFAULT_ARM_JOINT_POS,
)
"""Pre-train gains. Same task-space gains as the UR5e recipe: the G1 arm is ~5x lighter, so the loop is if
anything faster; the torque clamp is the per-joint motor limit (25 N m, wrists 5 N m)."""

G1D_DEX1_RELATIVE_OSC_EVAL = G1D_DEX1_RELATIVE_OSC.replace(
    scale_xyz_axisangle=(0.01, 0.01, 0.002, 0.02, 0.02, 0.2),
    motion_stiffness=(1000.0, 1000.0, 1000.0, 50.0, 50.0, 50.0),
    motion_damping_ratio=(1.0, 1.0, 1.0, 1.0, 1.0, 1.0),
)
```

3 Stage 1: Learning the Task

Stage 1 exists to learn the task, not to match the hardware. The simulator is kept as clean as the task allows, so that the policy receives a consistent learning signal while it discovers how to grasp, align and insert. The configuration is the base environment with the training events and the training action set. On the UR5e the robot is the implicit variant throughout the scene definition. On the G1-D the scene also selects the implicit variant, and the environment classes are one-line specialisations of the UR5e ones.

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/config/ur5e_robotiq_2f85/rl_state_cfg.py lines 728–747

```
# Training configuration (Stage 1: no curriculum, implicit actuator, no sysid DR)
@configclass
class Ur5eRobotiq2f85RelCartesianOSCTrainCfg(Ur5eRobotiq2f85RlStateCfg):

    events: TrainEventCfg = TrainEventCfg()
    actions: Ur5eRobotiq2f85RelativeOSCAction = Ur5eRobotiq2f85RelativeOSCAction()


# Finetune configuration (Stage 2: explicit actuator, curriculum ramps sysid + gains + scales)
@configclass
class Ur5eRobotiq2f85RelCartesianOSCFinetuneCfg(Ur5eRobotiq2f85RlStateCfg):
    """Finetune config: loads converged Stage 1 policy, explicit actuator from start, curriculum ramps DR."""

    events: FinetuneEventCfg = FinetuneEventCfg()
    actions: Ur5eRobotiq2f85RelativeOSCAction = Ur5eRobotiq2f85RelativeOSCAction()
    curriculum: FinetuneCurriculumsCfg = FinetuneCurriculumsCfg()

```

```
    def __post_init__(self):
        super().__post_init__()
        self.scene.robot = EXPLICIT_UR5E_ROBOTIQ_2F85.replace(prim_path="{ENV_REGEX_NS}/Robot")
```

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/config/g1d_dex1/rl_state_cfg.py lines 202–223

```
@configclass
class G1dDex1RelCartesianOSCTrainCfg(G1dDex1RlStateCfg):
    """Stage 1 training: implicit actuator, soft gains, four reset-state distributions."""

    events: G1dTrainEventCfg = G1dTrainEventCfg()
    actions: G1dDex1RelativeOSCAction = G1dDex1RelativeOSCAction()


@configclass
class G1dDex1RelCartesianOSCEvalCfg(G1dDex1RlStateCfg):
    """Evaluation after stage 1: ObjectAnywhereEEAnywhere resets only."""

    events: G1dTrainEvalEventCfg = G1dTrainEvalEventCfg()
    actions: G1dDex1RelativeOSCAction = G1dDex1RelativeOSCAction()


@configclass
class G1dDex1RelCartesianOSCStiffEvalCfg(G1dDex1RlStateCfg):
    """Evaluation with the stiff (end-of-curriculum) gains and small action scale."""

    events: G1dTrainEvalEventCfg = G1dTrainEvalEventCfg()
    actions: G1dDex1RelativeOSCEvalAction = G1dDex1RelativeOSCEvalAction()
```

3.1 What is randomised, and what is not

The training events are the shared base events plus the reset-state sampler. The base events randomise the things that belong to the *task*: the friction of the robot, the peg, the hole and the table, the masses of the same bodies, and the gripper actuator gains. They deliberately exclude the things that belong to the *motor*: the arm's joint friction, armature and delay, and the OSC gains. The docstring of the base event class says so explicitly, and the training event class only adds the reset sampler.

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/config/ur5e_robotiq_2f85/rl_state_cfg.py lines 237–255

```
@configclass
class TrainEventCfg(BaseEventCfg):
    """Training events: material/mass randomization + 4-path resets. No sysid or OSC gain
     randomization."""

    reset_from_reset_states = EventTerm(
        func=task_mdp.MultiResetManager,
        mode="reset",
        params={
            "dataset_dir": f"{UWLAB_CLOUD_ASSETS_DIR}/Datasets/OmniReset_isaaclab3",
            "reset_types": [
                "ObjectAnywhereEEAnywhere",
                "ObjectRestingEEGrasped",
                "ObjectAnywhereEEGrasped",
                "ObjectPartiallyAssembledEEGrasped",
            ],
            "probs": [0.25, 0.25, 0.25, 0.25],
            "success": "env.reward_manager.get_term_cfg('progress_context').func.success",
        },
    )
```

The reset sampler draws each episode's starting state from one of four pre-recorded distributions with equal probability. These range from "object anywhere, end effector anywhere" to "object partially assembled, end effector grasping", so the policy practises every phase of the insertion from the first iteration. The reset-state tutorial in this folder describes how those datasets are produced.

3.2 The controller settings

The action set is the soft one. The translational stiffness is 200 N/m, the rotational stiffness 3 N m/rad, and one unit of policy output moves the target by 2 cm or by 0.02 rad about x and y and 0.2 rad

about z. The UR5e uses a damping ratio of 3 on translation, which is strongly overdamped, and 1 on rotation. The G1-D uses zero for the reason given in Section 2.5.

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/config/ur5e_robotiq_2f85/actions.py

lines 15–33

```
UR5E_ROBOTIQ_2F85_RELATIVE_OSC = RelCartesianOSCActionCfg(
    asset_name="robot",
    joint_names=["shoulder.*", "elbow.*", "wrist.*"],
    body_name="wrist_3_link",
    scale_xyz_axisangle=(0.02, 0.02, 0.02, 0.02, 0.02, 0.2),
    motion_stiffness=(200.0, 200.0, 200.0, 3.0, 3.0, 3.0),
    motion_damping_ratio=(3.0, 3.0, 3.0, 1.0, 1.0, 1.0),
    torque_limit=(150.0, 150.0, 150.0, 28.0, 28.0, 28.0),
)

# Eval / sim2real gains (high Kp matched to sysid friction, end-of-curriculum values)
UR5E_ROBOTIQ_2F85_RELATIVE_OSC_EVAL = RelCartesianOSCActionCfg(
    asset_name="robot",
    joint_names=["shoulder.*", "elbow.*", "wrist.*"],
    body_name="wrist_3_link",
    scale_xyz_axisangle=(0.01, 0.01, 0.002, 0.02, 0.02, 0.2),
    motion_stiffness=(1000.0, 1000.0, 1000.0, 50.0, 50.0, 50.0),
    motion_damping_ratio=(1.0, 1.0, 1.0, 1.0, 1.0, 1.0),
    torque_limit=(150.0, 150.0, 150.0, 28.0, 28.0, 28.0),
```

Inside the action term these numbers become per-environment tensors. The per-environment copy is what allows the stage 2 events to change the gains of each environment independently without touching the configuration.

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/mdp/actions/task_space_actions.py

lines 64–73. 94–95

```
# Controller gains (per-env for domain randomization): Kd = 2 * sqrt(Kp) * damping_ratio
kp = torch.tensor(cfg.motion_stiffness, device=self.device, dtype=torch.float32)
damping_ratio = torch.tensor(cfg.motion_damping_ratio, device=self.device, dtype=torch.float32)
kd = 2.0 * torch.sqrt(kp) * damping_ratio
# Store defaults (1D) and expand to per-env (N, 6)
self._kp_default = kp
self._kd_default = kd
self._damping_ratio_default = damping_ratio
self._kp = kp.unsqueeze(0).expand(self.num_envs, -1).clone()
self._kd = kd.unsqueeze(0).expand(self.num_envs, -1).clone()

.
.
.
# Action scaling
self._scale = torch.tensor(cfg.scale_xyz_axisangle, device=self.device, dtype=torch.float32)
```

3.3 The learner

The learner is proximal policy optimisation (PPO) [4] from RSL-RL [5]. Table 2 lists the settings that matter for reading the curriculum timings later. The G1-D runner inherits all of them and changes only the experiment name.

Table 2: PPO runner settings shared by both robots.

Setting	Value	Meaning
policy rate	10 Hz	decimation 12 at a physics step of $1/120$ s
rollout length	32 steps per env	one PPO iteration is 32 policy steps, or 3.2 s of simulated time per environment
learning rate	10^{-4}, adaptive	adjusted to hold a KL divergence of 0.01 between updates
discount, GAE	0.99, 0.95	horizon of about 100 policy steps
entropy coefficient	0.006	exploration bonus
network	MLP 512, 256, 128, 64, ELU	actor and critic, with observation normalisation
action distribution	heteroscedastic Gaussian	state-dependent log standard deviation in $[0.001, 2]$

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/config/ur5e_robotiq_2f85/agents/rsl_rl_cfg.py lines 17–24. 47–60

```
@configclass
class Base_PPORunnerCfg(RslRlOnPolicyRunnerCfg):
    num_steps_per_env = 32
    max_iterations = 40000
    save_interval = 100
    obs_groups = {"actor": ["policy"], "critic": ["policy"]}
    resume = False
    experiment_name = "ur5e_robotiq_2f85_omnireset_agent"
```

```
    algorithm = RslRlPpoAlgorithmCfg(
        value_loss_coef=1.0,
        use_clipped_value_loss=True,
        normalize_advantage_per_mini_batch=False,
        clip_param=0.2,
        entropy_coef=0.006,
        num_learning_epochs=5,
        num_mini_batches=4,
        learning_rate=1.0e-4,
        schedule="adaptive",
        gamma=0.99,
        lam=0.95,
        desired_kl=0.01,
        max_grad_norm=1.0,
```

4 Stage 2: Moving the Policy Toward the Real Robot

A policy that has converged in stage 1 has never felt joint friction, has never waited for a delayed torque, and has only ever pushed with soft gains and large steps. Each of these is different on the hardware. Stage 2 closes the gap by continuing training while the simulator is moved, a little at a time, from the stage 1 plant to the real one. It is a fine-tuning stage because it starts from the stage 1 checkpoint rather than from random weights.

4.1 The configuration

The fine-tune environment differs from the training environment in three declarations. The events are the fine-tune events, which add two randomisation terms. The curriculum is no longer empty. And after the base initialisation, the scene's robot is replaced by the explicit variant, so the delayed actuator is in place from the first step of stage 2. The listing of the two environment classes in Section 3 shows all three.

The stage 1 checkpoint is loaded through the training script's resume mechanism. Passing a checkpoint path makes the runner load the weights before the first iteration, and training then continues under the fine-tune environment.

scripts/reinforcement_learning/rsl_rl/train.py lines 179–184. 210–214

```
# save resume path before creating a new log_dir
if args_cli.resume_path is not None:
    resume_path = retrieve_file_path(args_cli.resume_path)
    agent_cfg.resume = True
elif agent_cfg.resume or agent_cfg.algorithm.class_name == "Distillation":
    resume_path = get_checkpoint_path(log_root_path, agent_cfg.load_run, agent_cfg.load_checkpoint)
```

```
# load the checkpoint
if agent_cfg.resume or agent_cfg.algorithm.class_name == "Distillation":
    print(f"[INFO]: Loading model checkpoint from: {resume_path}")
    # load previously trained model
    runner.load(resume_path)
```

4.2 The progress variable and what it controls

Everything that stage 2 changes about the simulator is driven by a scalar progress variable $p \in [0, 1]$. At $p = 0$ the simulator is the stage 1 simulator with the explicit actuator. At $p = 1$ it is the evaluation

simulator. The progress variable is stored on the event terms and read every time they run, which is at every environment reset. Three families of quantities depend on it.

Arm dynamics from system identification. The calibrated armature a_j^{id}, static friction $\mu_{s,j}^{\mathrm{id}}$, dynamic-to-static ratio ρ_j^{id} and viscous friction b_j^{id} of each joint j are read from the robot's metadata file [6]. At each reset of an environment, a fresh scale $s \sim U(0.8, 1.2)$ is drawn per joint and per quantity, and the values written into PhysX are

$$\begin{aligned} a_j &= (1-p)\, a_j^0 + p\, s\, a_j^{\mathrm{id}}, \\ \mu_{s,j} &= p\, s\, \mu_{s,j}^{\mathrm{id}}, \qquad \mu_{d,j} = \min\big(s\, \rho_j^{\mathrm{id}}\, \mu_{s,j},\ \mu_{s,j}\big), \\ b_j &= p\, s\, b_j^{\mathrm{id}}, \\ d &\sim U\{0, \dots, \mathrm{round}(p\, d_{\max})\}, \quad d_{\max} = 1, \end{aligned} \tag{4}$$

where a_j^0 is the armature the asset started with and d is the motor delay in physics steps. Friction starts at exactly zero and the delay starts at exactly zero, so the $p = 0$ plant is frictionless and instantaneous, as in stage 1.

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/mdp/events.py lines 1701–1734

```
p = self.scale_progress

def _scale(nominal):
    val = torch.as_tensor(nominal, device=device, dtype=torch.float32)
    return val * (lo + torch.rand(N, n_joints, device=device) * (hi - lo))

# Armature and friction: interpolate from the startup model to sysid
if self._initial_armature is None:
    self._initial_armature = self.robot.data.joint_armature.torch[:, self.joint_ids].clone()
arm_vals = self._initial_armature[env_ids] * (1.0 - p) + _scale(self.armature) * p
sfric_vals = _scale(self.static_friction) * p
dratio_vals = _scale(self.dynamic_ratio) * p
dfric_vals = torch.minimum(dratio_vals * sfric_vals, sfric_vals)
vfric_vals = _scale(self.viscous_friction) * p

self.robot.write_joint_armature_to_sim_index(armature=arm_vals, joint_ids=self.joint_ids, env_ids=
    env_ids)
self.robot.write_joint_friction_coefficient_to_sim_index(
    joint_friction_coeff=sfric_vals,
    joint_dynamic_friction_coeff=dfric_vals,
    joint_viscous_friction_coeff=vfric_vals,
    joint_ids=self.joint_ids,
    env_ids=env_ids,
)

# Motor delay scaled by ADR progress (if actuator supports it)
delay_lo, delay_hi = delay_range
actuator = self.robot.actuators[self.actuator_name]
if hasattr(actuator, "positions_delay_buffer"):
    effective_hi = int(round(p * delay_hi))
    effective_lo = min(delay_lo, effective_hi)
    delays = torch.randint(effective_lo, effective_hi + 1, (N,), device=device, dtype=torch.int)
    actuator.positions_delay_buffer.set_time_lag(delays, env_ids)
    actuator.velocities_delay_buffer.set_time_lag(delays, env_ids)
    actuator.efforts_delay_buffer.set_time_lag(delays, env_ids)
```

Controller gains. The same event mechanism moves the OSC gains of each environment from the soft training values K_p^0, ζ^0 toward randomised copies of the terminal values K_p^1, ζ^1. Separate scales are drawn for the translational and rotational blocks and for stiffness and damping ratio, so four scalars per environment in total:

$$\begin{aligned} K_p(p) &= K_p^0 + p\,\big(s\, K_p^1 - K_p^0\big), \\ \zeta(p) &= \zeta^0 + p\,\big(s'\, \zeta^1 - \zeta^0\big), \\ K_d &= 2\,\zeta(p)\sqrt{K_p(p)}. \end{aligned} \tag{5}$$

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/mdp/events.py

lines 1862–1867. 1872–1888. 1898–1899

```
p = self.scale_progress

s_xyz = lo + torch.rand(n, 1, device=env.device) * (hi - lo)
s_rpy = lo + torch.rand(n, 1, device=env.device) * (hi - lo)
s_dr_xyz = lo + torch.rand(n, 1, device=env.device) * (hi - lo)
s_dr_rpy = lo + torch.rand(n, 1, device=env.device) * (hi - lo)
```

:

```
if terminal_kp is not None and terminal_damping_ratio is not None:
    # Terminal Kp with randomization
    kp_term = torch.tensor(terminal_kp, device=env.device, dtype=torch.float32)
    target_kp = kp_term.unsqueeze(0).repeat(n, 1)
    target_kp[:, :3] *= s_xyz
    target_kp[:, 3:] *= s_rpy

    dr_term = torch.tensor(terminal_damping_ratio, device=env.device, dtype=torch.float32)
    target_dr = dr_term.unsqueeze(0).repeat(n, 1)
    target_dr[:, :3] *= s_dr_xyz
    target_dr[:, 3:] *= s_dr_rpy

    # Interpolate from action defaults (initial) to terminal
    init_kp = kp_default.unsqueeze(0)
    init_dr = dr_default.unsqueeze(0)
    new_kp = init_kp + p * (target_kp - init_kp)
    new_dr = init_dr + p * (target_dr - init_dr)
```

:

```
self._action_term._kp[env_ids] = new_kp
self._action_term._kd[env_ids] = 2.0 * torch.sqrt(new_kp) * new_dr
```

Action scale. The per-step displacement that one unit of policy output commands shrinks linearly from the training scale σ^0 to the terminal scale σ^1. This one is applied to the action term directly and is shared by all environments:

$$\begin{aligned} \sigma(p) &= \sigma^0 + p\left(\sigma^1 - \sigma^0\right), \\ \sigma^0 &= (2,2,2)\,\mathrm{cm}, \qquad \sigma^1 = (1,1,0.2)\,\mathrm{cm}. \end{aligned} \tag{6}$$

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/mdp/events.py lines 2084–2096

```
success_rates = self._reset_term.success_monitor.get_success_rate()
mean_success = success_rates.mean().item()

if mean_success > success_threshold_up:
    self._progress = min(1.0, self._progress + delta)
elif mean_success < success_threshold_down:
    self._progress = max(0.0, self._progress - delta)

initial = torch.tensor(initial_scales, device=env.device, dtype=torch.float32)
target = torch.tensor(target_scales, device=env.device, dtype=torch.float32)
effective = initial + self._progress * (target - initial)

self._action_term._scale = effective
```

The fine-tune events and the terminal values are declared together. The curriculum class names the event terms it drives and sets the thresholds of the update rule.

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/config/ur5e_robotiq_2f85/rl_state_cfg.py lines 318–352

```
@configclass
class FinetuneEventCfg(TrainEventCfg):
    """Finetune events: curriculum-ramped sysid + OSC gains + 4-path resets. Explicit actuator from start."""

    randomize_arm_sysid = EventTerm(
        func=task_mdp.randomize_arm_from_sysid,
        mode="reset",
        params={
            "asset_cfg": SceneEntityCfg("robot"),
            "joint_names": [
                "shoulder_pan_joint",
                "shoulder_lift_joint",
                "elbow_joint",
```

```
                "wrist_1_joint",
                "wrist_2_joint",
                "wrist_3_joint",
            ],
            "actuator_name": "arm",
            "scale_range": (0.8, 1.2),
            "delay_range": (0, 1),
            "initial_scale_progress": 0.0,
        },
    )

    randomize_osc_gains = EventTerm(
        func=task_mdp.randomize_rel_cartesian_osc_gains,
        mode="reset",
        params={
            "action_name": "arm",
            "scale_range": (0.8, 1.2),
            "terminal_kp": (1000.0, 1000.0, 1000.0, 50.0, 50.0, 50.0),
            "terminal_damping_ratio": (1.0, 1.0, 1.0, 1.0, 1.0, 1.0),
            "initial_scale_progress": 0.0,
        },
    )
```

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/config/ur5e_robotiq_2f85/rl_state_cfg.py lines 578–610

```
@configclass
class FinetuneCurriculumsCfg:
    """Finetune curriculum: ADR sysid + action scale ramp. No actuator swap (explicit from start)."""

    adr_sysid = CurrTerm(
        func=task_mdp.adr_sysid_curriculum,
        params={
            "event_term_names": ["randomize_arm_sysid", "randomize_osc_gains"],
            "reset_event_name": "reset_from_reset_states",
            "success_threshold_up": 0.95,
            "success_threshold_down": 0.9,
            "delta": 0.01,
            "update_every_n_steps": 200,
            "initial_scale_progress": 0.0,
            "warmup_success_threshold": 0.95,
        },
    )

    action_scale = CurrTerm(
        func=task_mdp.action_scale_curriculum,
        params={
            "action_name": "arm",
            "reset_event_name": "reset_from_reset_states",
            "initial_scales": [0.02, 0.02, 0.02, 0.02, 0.02, 0.2],
            "target_scales": [0.01, 0.01, 0.002, 0.02, 0.02, 0.2],
            "success_threshold_up": 0.95,
            "success_threshold_down": 0.9,
            "delta": 0.01,
            "update_every_n_steps": 200,
            "initial_progress": 0.0,
        },
    )
```

4.3 The update rule

The progress variable is not advanced on a timer. It is advanced by the policy's own success rate, so the simulator only becomes harder when the policy has shown that it can cope with the current difficulty. The rule is a simple hysteresis controller. Let $\bar{s}$ be the mean success rate across the reset-state distributions, as reported by the reset sampler's success monitor. Every 200 environment steps,

$$p \leftarrow \begin{cases} \min(1,\ p + 0.01) & \text{if } \bar{s} > 0.95, \\ \max(0,\ p - 0.01) & \text{if } \bar{s} < 0.90, \\ p & \text{otherwise.} \end{cases} \tag{7}$$

The band between 0.90 and 0.95 is where the policy is allowed to recover without the difficulty changing. A separate warm-up gate holds $p = 0$ until the success rate has reached 0.95 at least once,

so that a freshly loaded checkpoint which performs poorly under the new actuator is given time before anything else changes. Once opened, the gate stays open.

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/mdp/events.py

lines 1977–1980. 1988–1989. 1991–1994. 2002–2011

```
# Only update once every N env steps (agnostic to num_envs)
current_step = env.common_step_counter
if (current_step - self._last_update_step) < update_every_n_steps:
    return self._last_state
```

:

```
success_rates = self._reset_term.success_monitor.get_success_rate()
mean_success = success_rates.mean().item()
```

:

```
# Warmup gate: hold scale_progress until success exceeds threshold
if not self._warmed_up:
    if mean_success >= self._warmup_threshold:
        self._warmed_up = True
```

:

```
# Update scale_progress based on thresholds
current_progress = self._event_terms[0].scale_progress if self._event_terms else 0.0
if mean_success > success_threshold_up:
    current_progress = min(1.0, current_progress + delta)
elif mean_success < success_threshold_down:
    current_progress = max(0.0, current_progress - delta)

# Apply to all target event terms
for term in self._event_terms:
    term.scale_progress = current_progress
```

There are two independent copies of this rule. One drives the arm dynamics and the OSC gains together, so a single progress value moves friction, delay and gains in lockstep. The other drives the action scale. Both read the same success rate and use the same thresholds and step, but each keeps its own counter and its own progress, so in practice they drift apart by a few hundredths. Figure 3 shows a schematic run of the rule.

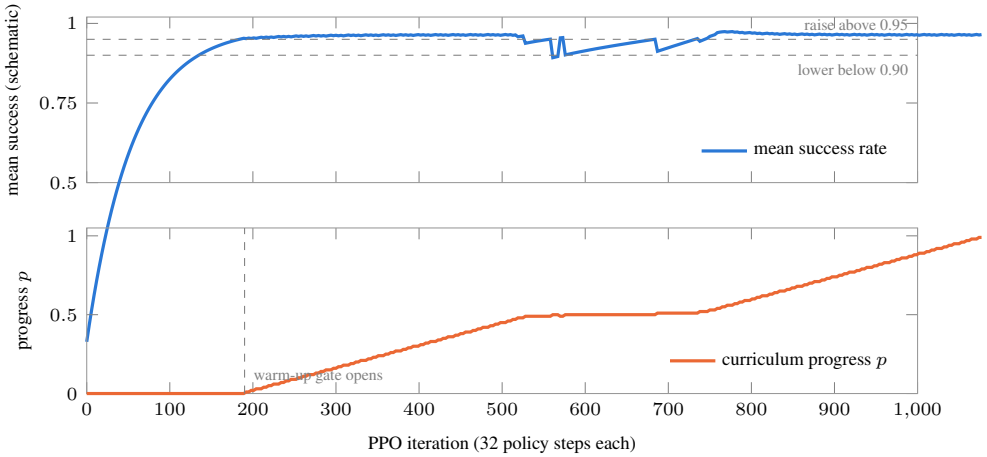


Figure 3: A schematic run of the update rule, produced by a toy model and not by a training log. The success rate rises as the policy learns, the warm-up gate opens when it first touches 0.95, and progress then climbs in steps of 0.01 whenever success stays above the upper threshold. Around the middle of the ramp the plant becomes noticeably harder, success falls below 0.90, progress retreats until the policy recovers, and the climb resumes. The ramp ends when progress reaches 1.

The timings follow from the numbers in Eq. (7). The counter is the environment's global step counter, which advances once per policy step regardless of how many parallel environments there are. Reaching $p = 1$ from 0 needs at least 100 raises, hence at least 20 000 policy steps. At 32 policy

steps per PPO iteration, that is at least 625 iterations, or 2 000 s of simulated time per environment. Any excursion below 0.90 adds to this, because every retreat has to be climbed again.

4.4 Why ramp rather than switch

One might ask why the plant is not simply switched to its final form at the start of stage 2. The reason is that the stage 1 policy is tuned to the stage 1 plant. Under stiff gains and real friction the same actions produce different motion, the observations drift out of the distribution the policy has seen, and the success rate collapses. PPO can recover from a collapse, but it does so slowly and sometimes to a worse policy. The ramp keeps the policy inside a regime where it mostly succeeds, so every update is made from informative, mostly successful rollouts. The success gate turns the ramp speed into a measured quantity rather than a guess.

5 The Evaluation Environments

There are two evaluation environments per robot, one per stage, and each reproduces the plant of the stage it evaluates. The evaluation after stage 1 uses the implicit actuator and the soft action set. The evaluation after stage 2 uses the explicit actuator, the stiff action set, and the fixed variants of the two stage 2 events. Both reduce the reset distribution to the single "object anywhere, end effector anywhere" case.

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/config/ur5e_robotiq_2f85/rl_state_cfg.py lines 750–769

```python
# Evaluation configuration (after Stage 1: implicit actuator, soft gains, no sysid DR)
@configclass
class Ur5eRobotiq2f85RelCartesianOSCEvalCfg(Ur5eRobotiq2f85RlStateCfg):
    """Eval after Stage 1: implicit actuator, soft gains, large action scale, no sysid DR."""

    events: TrainEvalEventCfg = TrainEvalEventCfg()
    actions: Ur5eRobotiq2f85RelativeOSCAction = Ur5eRobotiq2f85RelativeOSCAction()


# Evaluation configuration (after Stage 2: explicit actuator, stiff gains, fixed sysid)
@configclass
class Ur5eRobotiq2f85RelCartesianOSCFinetuneEvalCfg(Ur5eRobotiq2f85RlStateCfg):
    """Eval after Stage 2: explicit actuator, stiff gains, small action scale, fixed sysid + OSC gains."""

    events: FinetuneEvalEventCfg = FinetuneEvalEventCfg()
    actions: Ur5eRobotiq2f85RelativeOSCEvalAction = Ur5eRobotiq2f85RelativeOSCEvalAction()

    def __post_init__(self):
        super().__post_init__()
        self.scene.robot = EXPLICIT_UR5E_ROBOTIQ_2F85.replace(prim_path="{ENV_REGEX_NS}/Robot")
```

The fixed event variants are subclasses of the stage 2 events that pin $p = 1$ and skip the curriculum. They still draw the $U(0.8, 1.2)$ scales at every reset, so the stage 2 evaluation sees the same spread of plants that the end of training saw, rather than one nominal plant. The stiff action set supplies the terminal gains and scale as the defaults that those scales multiply.

source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/config/ur5e_robotiq_2f85/rl_state_cfg.py lines 274–304

```python
@configclass
class FinetuneEvalEventCfg(BaseEventCfg):
    """Eval after Stage 2: fixed sysid + OSC gains (scale_progress=1) + 1-path resets."""

    randomize_arm_sysid = EventTerm(
        func=task_mdp.randomize_arm_from_sysid_fixed,
        mode="reset",
        params={
            "asset_cfg": SceneEntityCfg("robot"),
            "joint_names": [
                "shoulder_pan_joint",
                "shoulder_lift_joint",
                "elbow_joint",
                "wrist_1_joint",
```

```
                "wrist_2_joint",
                "wrist_3_joint",
            ],
            "actuator_name": "arm",
            "scale_range": (0.8, 1.2),
            "delay_range": (0, 1),
        },
    )

    randomize_osc_gains = EventTerm(
        func=task_mdp.randomize_rel_cartesian_osc_gains_fixed,
        mode="reset",
        params={
            "action_name": "arm",
            "scale_range": (0.8, 1.2),
        },
    )
```

6 Why Training and Evaluation Differ

Table 1 showed three controller parameters that differ between the training and evaluation action sets: the stiffness, the damping ratio and the action scale. None of them is a separate design. The evaluation values are the end of the stage 2 ramp, and the training values are its start. The question is therefore why the ramp exists at all, parameter by parameter.

6.1 Stiffness: friction sets a floor on the useful gain

Consider the controller holding the end effector against a resisting force. At rest the task-space PD law reduces to $F = K_p\, e$, so a resisting force F_f that the controller must overcome leaves a steady-state error

$$e_{\mathrm{ss}} = \frac{F_f}{K_p}. \tag{8}$$

On the real arm, joint friction and the stiction of a peg in a hole both act as such a force. Suppose the friction of the wrist joints is equivalent to 5 N at the end effector, which is an illustrative figure rather than a measurement. At $K_p = 200$ N/m the controller stops 25 mm short of its target. At $K_p = 1000$ N/m it stops 5 mm short. For a peg with a fraction of a millimetre of clearance, the first number makes the task impossible and the second makes it merely hard. The stiff gain is the one that the real robot's controller runs, chosen against the friction found by system identification. The soft gain exists only because it is easier to learn under, as Section 6.4 explains.

6.2 Damping ratio: overdamped for exploration, critical for deployment

The UR5e trains with a translational damping ratio of 3, which is strongly overdamped. An overdamped loop approaches its target slowly and never overshoots, so a random early policy cannot make the arm swing through the table or the hole. The evaluation ratio of 1 is critically damped, which is the fastest response without overshoot, and is what the hardware controller uses. The G1-D trains with no explicit damping at all, for the stability reason in Section 2.5, which makes its jump from training to evaluation damping larger than the UR5e's.

6.3 Action scale: how far the arm moves per policy step

With soft gains the controller only ever reaches a fraction of a 2 cm step within the 0.1 s policy period, so the effective motion is gentle even though the commanded step is large. With stiff gains the step is reached within the period, and the step size becomes the speed:

$$v = \frac{\sigma}{\Delta t_{\mathrm{policy}}} = \begin{cases} 0.2\,\mathrm{m/s} & \text{at } \sigma = 2\,\mathrm{cm}, \\ 0.1\,\mathrm{m/s} & \text{at } \sigma = 1\,\mathrm{cm}, \\ 0.02\,\mathrm{m/s} & \text{at } \sigma = 2\,\mathrm{mm}. \end{cases} \tag{9}$$

Shrinking the scale as the gains stiffen keeps the end-effector speed in a range that insertion tolerates, and the 2 mm vertical scale gives the policy millimetre control over the one axis that needs it. The

alternative of clipping the pose error inside the controller would saturate the PD law and flatten the relationship between action and motion. Scaling the action instead keeps that relationship linear, which is what a policy gradient needs.

6.4 Why not train stiff from the start

Put the three together and the ordering is forced. A stiff controller with a 2 cm step applies a 20 N task force to every step, against 4 N for the soft one. An untrained policy emits random 2 cm steps ten times a second. Stiff gains therefore mean hard contacts, frequent collisions and many episodes that end in a termination before any reward is collected. Soft gains make exploration survivable. Once the policy knows what to do, the gains can be raised and the steps shortened, and the policy adapts to the changed response because it observes the end-effector pose and velocity rather than its own commands.

7 The G1-D Today

The G1-D configuration implements stage 1 and its evaluation. It does not yet implement stage 2. Table 3 lists what exists for each robot.

Table 3: Environment classes per robot.

Role	UR5e + Robotiq 2F-85	G1-D + Dex1-1
stage 1 train	`Ur5eRobotiq2f85RelCartesianOSCTrainCfg`	`G1dDex1RelCartesianOSCTrainCfg`
eval after stage 1	`Ur5eRobotiq2f85RelCartesianOSCEvalCfg`	`G1dDex1RelCartesianOSCEvalCfg`
stage 2 fine-tune	`Ur5eRobotiq2f85RelCartesianOSCFinetuneCfg`	none
eval after stage 2	`Ur5eRobotiq2f85RelCartesianOSC FinetuneEvalCfg`	`G1dDex1RelCartesianOSCStiffEvalCfg` (partial, see below)

Three observations follow from reading the configuration against the UR5e recipe. They are listed as facts about the current code, not as a plan.

1. **The stiff evaluation evaluates an untrained regime.** The G1-D stiff evaluation config applies the terminal gains and scale to a stage 1 policy, but with the implicit actuator and without the fixed sysid events. A stage 1 policy has trained at $K_p = 200$ with $\zeta = 0$ and is here evaluated at $K_p = 1000$ with $\zeta = 1$. A lower success rate than the plain evaluation is expected, and the gap is a measure of how much a stage 2 would change rather than a defect.
2. **The explicit G1-D actuator has no joint damping.** The implicit variant carries the 1.0 N m s/rad that replaces the OSC's explicit damping. The explicit variant, which a stage 2 would use, sets damping to zero, and the terminal damping ratio of 1 would re-introduce the explicit task-space damping of about 14 N m s/rad that Section 2.5 argues is unstable on this wrist.
3. **There is no system identification for the G1-D.** The sysid event reads an `armature`, `static_friction`, `dynamic_ratio` and `viscous_friction` block from the robot's metadata file. The G1-D metadata file has no such block, so the UR5e fine-tune events cannot be instantiated against it as they stand.

8 Where Everything Lives

All paths are relative to the repository root. The task package root `source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset` is abbreviated to `omnireset/`.

Table 4: File index.

What	Where
implicit, ideal and delayed PD actuators	Isaac Lab 17.0.2, `isaaclab/actuators/actuator_pd.py`
UR5e articulation, implicit and explicit variants	`source/uwlab_assets/uwlab_assets/robots/ur5e_robotiq_gripper/ur5e_robotiq_2f85_gripper.py`
G1-D articulation, implicit and explicit variants	`source/uwlab_assets/uwlab_assets/robots/unitree_g1d_dex1/g1d_dex1.py`
OSC action term, per-environment gains	`omnireset/mdp/actions/task_space_actions.py`
UR5e action sets, soft and stiff	`omnireset/config/ur5e_robotiq_2f85/actions.py`
G1-D action sets, soft and stiff	`omnireset/config/g1d_dex1/actions.py`
UR5e environments, events and curriculum	`omnireset/config/ur5e_robotiq_2f85/rl_state_cfg.py`
G1-D environments and events	`omnireset/config/g1d_dex1/rl_state_cfg.py`
sysid, gain and action-scale events, curriculum terms	`omnireset/mdp/events.py`
PPO runner	`omnireset/config/ur5e_robotiq_2f85/agents/rsl_rl_cfg.py`
checkpoint loading	`scripts/reinforcement_learning/rsl_rl/train.py`, flag `-resume_path`
registered task names	`OmniReset-Ur5eRobotiq2f85-RelCartesianOSC-State-{,Finetune-,Play-,Finetune-Play-}v0`, `OmniReset-G1dDex1-RelCartesianOSC-State-{,Play-}v0`

References

[1] O. Khatib. A unified approach for motion and force control of robot manipulators: The operational space formulation. *IEEE Journal on Robotics and Automation*, 3(1):43–53, 1987.

[2] UW Lab. Emergent dexterity via diverse resets and large-scale reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2603.15789.

[3] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, and A. Garg. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023.

[4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[5] N. Rudin, D. Hoeller, P. Reist, and M. Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Conference on Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, pages 91–100, 2022.

[6] F. Bjelonic et al. PACE: Physics-aware calibration for sim-to-real transfer. *arXiv preprint arXiv:2509.06342*, 2025.